

ドイツ語学習アプリ 開発レポート

今回のセッションで何を作り、何が起きて、
どう解決したのかを、Pythonの知識をベースに解説します。

対象アプリ: Deutsch Lernen (ドイツ語基礎学習アプリ)

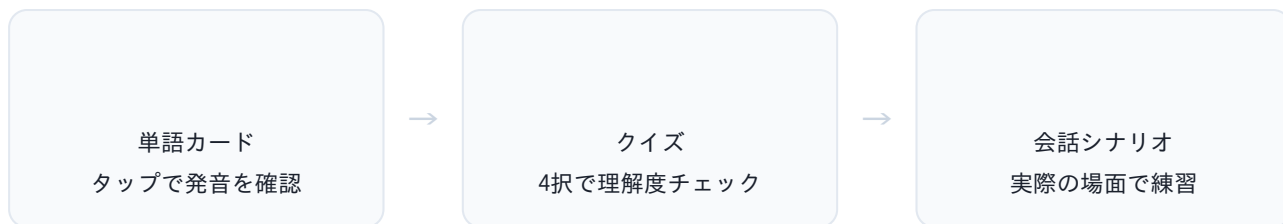
リポジトリ: diodiodiio/1st_test

ブランチ: claude/german-learning-app-9ijhw7

作成日: 2026年7月26日

1 今回作ったものの全体像

渡航前にドイツ語の基礎を身につけるためのスマートフォンアプリを、ゼロから作りました。「単語カード → クイズ → 会話シナリオ」の3ステップで1レッスンが進み、XPとストリーク（連続学習日数）で続けたい工夫を入れています。



このレポートでは、単に「機能を作った」だけでなく、実際にアプリを動かして問題を見つけ、直す というソフトウェア開発でとても重要な工程まで含めて、何をしたのかを順を追って説明します。

このレポートの読み方

あなたはPythonの基礎をご存知とのことなので、随所で「Pythonだとこれに近い」という対比を入れています。完全に一致する概念ではありませんが、イメージを掴む助けにしてください。

2 なぜこの技術を選んだか

スマートフォンアプリを作る場合、Pythonは主流の選択肢ではありません。iPhoneやAndroidはそれぞれ独自の開発言語（Swift、Kotlinなど）を持っていますが、その差を吸収して1つのコードでiPhoneでもAndroidでもブラウザでも動くアプリを作れる仕組みとして、今回は React Native（JavaScript/TypeScript）と、その開発を楽にする Expo というツールセットを使いました。

概念	今回使ったもの	Pythonでの近い概念
プログラミング言語	TypeScript (JavaScriptに型を付けたもの)	Python (型ヒント付き)
アプリの土台	React Native + Expo	Flask / Django のような「枠組み」
画面の部品化	React コンポーネント (関数)	再利用可能な関数やクラス
パッケージ管理	npm と package.json	pip と requirements.txt
依存ライブラリの実体	node_modules/ フォルダ	venv/ の中身に近い
実行コマンド	npx expo start	python manage.py runserver

つまり「Pythonで言うところの Flask/Django にあたるものが React Native/Expo」で、「pip にあたるものが npm」だとイメージすると全体の構造がつかみやすくなります。

3 プロジェクトの中身

作成したファイルは、役割ごとにフォルダを分けています。Pythonプロジェクトで `models/`、`views/`、`utils/` のようにフォルダを分けるのと同じ発想です。

```

1st_test/ └─ App.tsx # アプリの一番の入り口 (main.py のようなもの) └─ package.json # 依存ライブラリの一覧 (requirements.txt 相当) └─ app.json # アプリの名前やアイコンなどの設定 └─
assets/ # アイコン・画像 └─ src/ └─ data/ | └─ types.ts # データの型定義 (class の設計図に近い) | └─ curriculum.ts # 全レッスンの中身 (単語・クイズ・会話文) └─ components/ # 再利用する部品 | └─ FlashCard.tsx # 単語カード1枚分の見た目と挙動 | └─ QuizCard.tsx # クイズ1問分の見た目と挙動 | └─ ConversationCard.tsx # 会話シナリオの表示 └─ screens/ # 1画面ずつ | └─
HomeScreen.tsx / CurriculumScreen.tsx | └─ LessonScreen.tsx / ResultScreen.tsx / ProfileScreen.tsx └─ context/ | └─ ProgressContext.tsx # 進捗・XP・ストリークの管理 └─
hooks/ | └─ useSpeech.ts # 音声読み上げ機能 └─ navigation/ └─ AppNavigator.tsx # 画面遷移のルール (urls.py に近い)

```

データはどう持っているか

6つのユニット (挨拶・数字・買い物…) とそのレッスン内容は、すべて `curriculum.ts` という1つのファイルに、Pythonの辞書のリストに近い形で書いています。

```
// TypeScript (今回のコード)
{
  german: "Hallo",
  pronunciation: "ハロー",
  japanese: "こんにちは",
}

# もしPythonの辞書で書くなら、こんなイメージ
{
  "german": "Hallo",
  "pronunciation": "ハロー",
  "japanese": "こんにちは",
}
```

つまり「新しい単語やレッスンを追加したい」だけなら、このファイルにデータを1個追加するだけで済む設計にしています。

4 アプリが「覚えている」仕組み

このアプリは「どのレッスンを終えたか」「XPは何点か」「今日は連続何日目か」を覚えて、アプリを閉じて再度開いても消えないようにしています。これを実現しているのが `ProgressContext.tsx` です。

今回のコードでの概念	Pythonでの近い概念
<code>useState</code> (状態)	関数の外に置いた変数で、値が変わるとUIも再描画される、という自動連携付きの変数
<code>AsyncStorage</code> (保存先)	ファイルに <code>json.dump()</code> して保存し、次回起動時に <code>json.load()</code> で読み込むイメージ
<code>Context</code> (全体共有)	複数の関数から参照できるグローバル変数を、安全に扱うための仕組み

なぜこの仕組みが必要か

ホーム画面にもプロフィール画面にも「XP」を表示する必要があります。Pythonでグローバル変数をあちこちで直接書き換えると、どこで値が変わったか追いにくなりバグの元になります。Reactの `Context` は「変更していい場所を1箇所に決めて、他はそれを読むだけにする」ルールを強制することで、この問題を防いでいます。

5 音声読み上げ機能の追加

「発音がわからないと実践で使えない」というご要望を受けて、`expo-speech` という ライブラリを追加しました。Pythonで言えば `pip install pyttsx3` のようなイメージで、「テキストを渡すと端末が声に出して読んでくれる」機能を追加するライブラリです。

```
// src/hooks/useSpeech.ts (今回作成)
function speak(text: string) {
  Speech.speak(text, {
    language: 'de-DE', // ドイツ語の発音で読む
    rate: 0.82,        // 少しゆっくり
  });
}
```

これを3箇所に組み込みました。

- ・ 単語カード：カードが表示されると自動で発音、 ボタンでも再生
- ・ クイズ：問題のドイツ語を自動読み上げ
- ・ 会話シナリオ：セリフごとの に加え、「▶ 全部聴く」で通して再生

6 あなたのPCで動かすまでに起きたこと

「ブラウザで実際に触ってみたい」というご要望を受けて、Windows PCで動かすまでを一緒に進めました。起きた出来事と、それぞれの意味を整理します。

① Git が入っていなかった

`git clone` はソースコードをまるごとコピーしてくる操作ですが、 そのための「git」というツール自体が入っていませんでした。Pythonで言えば `pip` コマンド自体が使えない状態に近いです。公式サイトからインストールして解決しました。

② npm install で依存ライブラリを取得

`package.json` に書かれた約900個のライブラリを、まとめてダウンロード・インストールする コマンドです。Pythonの `pip install -r requirements.txt` に相当します。

③ npx expo start --web でサーバー起動

これは「開発用サーバーを起動して、ブラウザからアクセスできるようにする」コマンドです。Pythonでいう `python manage.py runserver` や Flask の `app.run()` と同じ役割で、起動すると `http://localhost:8081` のようなアドレスが使えるようになります。

④ アイコン画像が見つからないエラー

- 設定ファイル（`app.json`）で「このアイコン画像を使う」と指定していたのに、実際の画像ファイルを用意し忘れていました。Pythonで言えば、コードの中で `open("icon.png")` と書いたのにそのファイルが存在しない状態（`FileNotFoundError`）と同じです。画像を作成して追加し、解決しました。

ここまでで分かること

「動かない」の多くは、コードそのものの間違いではなく「必要な道具（Git）」「必要な材料（画像ファイル）」が足りていないだけという原因がとても多いです。エラーメッセージを 読むと、大抵「何が足りないか」が書かれています。

7 実際に見つけて直したバグ

「触ってみて確認したい」というご要望に応え、私自身がブラウザを自動操作してアプリの中を 実際に通り動かしました。その過程で、本物のバグを1つ見つけて修正しています。これは実際の開発現場でもとても大事な工程です。

見つけたバグ：クイズの答えが次の問題に引き継がれる

何が起きていたか

クイズで1問目に正解した状態のまま2問目に進むと、2問目がまだ何も答えていないのに「既に不正解として選択済み」の状態が表示されてしまうことがありました。1問目の正解の選択肢（「こんにちは」）が、たまたま2問目の不正解の選択肢としても存在していたために起きた問題です。

原因をPython風の擬似コードで表すと、こういう状態でした。

```
# 問題ごとに新しく作られるはずが、実際は使い回されていた
selected_answer = None    # 1問目の答えを保存する変数

# 1問目で "こんにちは" を選択 → selected_answer = "こんにちは"
# 2問目に進んでも selected_answer がリセットされず
# "こんにちは" のままだった（本来は None に戻るべき）
```

修正内容：問題が切り替わったタイミングで、選択状態を明示的にリセットする処理を1行追加しました。

```
// src/components/QuizCard.tsx
useEffect(() => {
  · setSelected(null); // ← この1行を追加
  // ...
}, [question.id]); // 問題が変わるたびに実行される
```

学びのポイント

この種のバグは、コードを目で読むだけでは気づきにくく、実際にアプリを動かして再現させて初めて見つかることが多いです。「動くはず」と「本当に動く」の間には 差があるということを覚えておく役立ちます。

もう一つの不具合：アイコン画像の欠如

前章で触れた「アイコン画像が見つからない」エラーも、実際にはコード上の設定（`app.json`）と実体（`assets/` フォルダの中身）が一致していなかったために起きたバグです。プレースホルダー画像を作成して解決しました。

8 変更の記録方法（バージョン管理）

コードを直すたびに、Git という仕組みで「いつ・何を・なぜ変えたか」を記録しています。

Gitの操作	意味
<code>git add</code>	「この変更を次の記録に含める」と選ぶ操作
<code>git commit</code>	選んだ変更に、説明文をつけて記録として確定する操作（セーブポイントを作るイメージ）
<code>git push</code>	手元の記録を、共有サーバー（GitHub）にアップロードする操作
branch（ブランチ）	本体に影響を与えずに作業できる「作業用の別ライン」。 今回は <code>claude/german-learning-app-9ijhw7</code> という名前で作業

今回、次の3回に分けてコミットしました。

1. 音声読み上げ機能の追加
2. クイズのバグ修正 + ブラウザ動作に必要な設定の追加

3. 不足していたアイコン画像の追加

なぜ細かく分けるか

「音声機能の追加」と「バグ修正」を1つのコミットにまとめてしまうと、後で「あのバグ修正だけを取り消したい」と思ったときに困ります。1つのコミット=1つの意味のある変更 にしておく、後から履歴を追いやすくなります。

9

これを自分のスキルにするために

Python基礎がある状態から、今回のような開発を自分でも進められるようになるための、おすすめの順番です。

STEP 1 JavaScriptの基礎文法

Pythonとの違い（`let / const`、`=>` 関数、`{ }` の使い方）に慣れる。「Python と対応させて覚える」で十分。

STEP 2 TypeScriptの型

Pythonの型ヒント（`def f(x: int) -> str:`）を知っていれば、考え方はほぼ同じ。

STEP 3 React の基本（コンポーネント・useState）

「画面の部品=関数」「状態が変わると自動で再描画される」という考え方に慣れる。

STEP 4 このアプリを実際に自分で改造してみる

まずは `src/data/curriculum.ts` に新しい単語を1つ追加してみる、 ボタンの色を変えてみる、といった小さな変更から。

STEP 5 Git操作を自分の手で行う

`git add` → `git commit` → `git push` を自分で打てるようになると、 AIに頼らず自分で変更を記録・共有できるようになります。

本レポートは、開発中に実施した対話・自動テスト・修正内容をもとに作成しています。